一、中断优先级

1.1 概念

中断优先级与中断屏蔽

■ **中断优先级**：CPU响应并处理中断请求的先后次序。

□ 计算机系统中一般包含**多个中断源**，当**同时**有多个中断源向CPU发出**中断请求**时，CPU应该首先响应哪一个中断请求，这取决于中断优先级：优先级高的先响应，优先级低的后响应。

□ 在**多重中断（嵌套中断）**中，高优先级中断请求可以中断CPU正在执行的低优先级中断服务程序（ISR）。

■ 综上所述，**中断优先级**包含两个层面的含义：**响应优先级**和**处理优先级**。

1.2 中断==响应优先级==和中断==处理优先级==

响应优先级

- **响应优先级**：CPU对同时来自多个中断源的中断请求进行响应的先后次序。
- 进行**硬件设计**时，**响应优先级就已固定**，难以更改。
- 响应优先级一般按照中断事件的**重要性**和**迫切性**来划分：
  ① 不可屏蔽中断 > 内部异常 > 可屏蔽中断。
  ② 在内部异常中，硬件终止（Abort）异常的优先级最高，其次是故障（Fault）或自陷（Trap）等软件异常。
  ③ DMA中断 > I/O设备传送中断。
  ④ 在I/O设备传送中断请求中，高速设备优先于低速设备，输入设备优先于输出设备，实时控制设备优先于普通设备。

处理优先级

- **处理优先级**：多重中断（嵌套中断）的实际优先级处理次序。
  - 在不使用中断屏蔽技术的情况下，处理优先级与响应优先级相同。
  - 可以利用**中断屏蔽技术**动态调整多重中断（嵌套中断）的实际优先级：使低优先级的中断也可以打断高优先级的中断服务（ISR），进而更加灵活地处理中断。
- 现代计算机中普遍采用中断屏蔽技术。

二、中断屏蔽

中断优先级与中断屏蔽

- 在中断控制器中包含**中断屏蔽寄存器**（Interrupt Mask Register，IMR）。
- IMR中的每一位对应一个设备中断源：
  - 为“1”时表示**屏蔽**对应设备发送的中断请求信号。
  - 为“0”时表示**允许**对应设备发送中断请求信号。
- IMR的值称为**中断屏蔽字**，通过给各设备配置中断屏蔽字，可以动态改变处理优先级的次序。

请注意：

- ✓ **中断屏蔽**仅对CPU运行中断服务程序ISR时的处理优先级（嵌套中断）起作用，而不会影响CPU运行主程序时的中断**响应优先级**。
- ✓ 若在CPU运行中断服务程序ISR时同时到达多个未屏蔽中断请求，**处理优先级**只能按**响应优先级**进行处理。
- ✓ IMR的值仅在执行中断服务程序ISR时才能被更新。

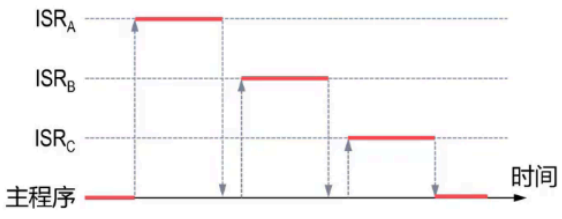
举例

中断优先级与中断屏蔽

【举例1】假设CPU在执行主程序时，设备A、B、C同时产生中断请求。

设备	响应优先级	中断屏蔽字（用于多重中断）		
		A	B	C
A	A>B>C	1	1	1
B		0	1	1
C		0	0	1

CPU运行轨迹

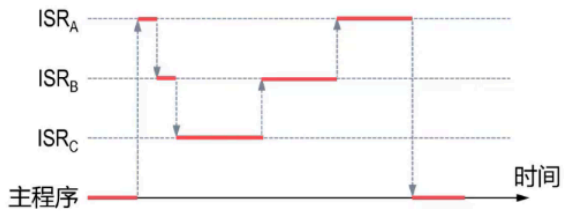


处理优先级：A>B>C，与**响应优先级**一致。

【举例2】假设CPU在执行主程序时，设备A、B、C同时产生中断请求。

设备	响应优先级	中断屏蔽字（用于多重中断）		
		A	B	C
A	A>B>C	1	0	0
B		1	1	0
C		1	1	1

CPU运行轨迹



处理优先级：C>B>A，与**响应优先级**相反。

从==深度优先搜索和回溯角度==理解中断屏蔽技术

表面上看，这是一个软件的运行过程，但**实际上，这种嵌套和回溯是由纯硬件（主要是CPU内部的中断控制器和寄存器）配合少量的指令来实现的**

把整个过程拆解为硬件中的三个核心机制：**请求锁存（记录要搜的节点）**、**屏蔽字（决定要不要往深处走）**、**堆栈（用来回溯的线索）**

机制一：为什么能同时知道 A、B、C 都来了？（中断请求寄存器 IRR）

当 A、B、C 同时产生中断请求时，它们各自向中断控制器（如经典的 8259A 或现代的高级可编程中断控制器 APIC）发送电信号

硬件中有一个**中断请求寄存器（IRR）**，它就像一个位图（Bitmask）。A、B、C 同时来，IRR 寄存器的对应位就会被硬件置为 `1`，例如变成 `111`

即使 CPU 正在执行主程序，这个寄存器也会一直**锁存**这些请求，保证请求不丢失。这就是 DFS 中"把所有相邻节点压入栈"的硬件基础

机制二：为什么 A 先响应，却被 B 抢占？（中断屏蔽字与优先级仲裁）

这就涉及到你图中的表格了。硬件中有两个非常关键的寄存器：

1. **中断屏蔽寄存器（IMR）**：就是图中的表格内容
2. **优先级仲裁器**：决定当前应该响应谁

在这张图的设定中：

* **响应优先级**：A > B > C（硬件固定死的，谁的数字小谁先上）
* **屏蔽字规则**：`1` 表示屏蔽（不准打断），`0` 表示开放（允许打断）

具体过程（DFS的深入过程）：

1. **CPU 最初**：执行主程序，IMR 可能是全 `0`（全开放）。A、B、C 同时请求。仲裁器一看，A 的硬件优先级最高，所以**先响应 A**
2. **进入 A 的处理程序**：此时，CPU 会**强制将表格中 A 对应的屏蔽字 `100` 写入 IMR 寄存器。**

* 这代表什么意思？代表"我现在在处理 A，我屏蔽我自己（第一位是1），但我**不屏蔽 B 和 C**（第二、三位是0）"
* 此时，B 和 C 的请求依然在 IRR 中挂着。因为 A 的屏蔽字允许 B 和 C 打断它，所以硬件立刻发现 B 的优先级高于 C，**于是暂停 A，跳转去执行 B。**这就形成了 DFS 的"深入"。

3. **进入 B 的处理程序**：CPU 将表格中 B 的屏蔽字 `110` 写入 IMR 寄存器

* 意思是："我屏蔽 A（之前被打断的，不能让它反客为主）和我自己，但**不屏蔽 C**"。
* 此时 C 立刻打断 B，CPU 跳转去执行 C。这就完成了最深一层的"深入"

4. **进入 C 的处理程序**：CPU 将表格中 C 的屏蔽字 `111` 写入 IMR

* 意思是："屏蔽 A、B、C（屏蔽字全部为1）"。此时，没有任何设备能打断 C 了，C 开始安心执行完自己的代码

机制三：处理完 C 后，怎么准确回到 B，再回到 A？（硬件堆栈 Stack）

这是实现"回溯"最巧妙的地方。CPU 内部有一组专门的寄存器（通常叫 `SP` 栈指针），并在内存中开辟了一块专门的空间叫**系统堆栈**

回溯过程：

1. 主程序被打断时，CPU 硬件自动将主程序的**返回地址**和**程序状态字（PSW）**压入堆栈。
2. A 被打断时，CPU 硬件自动将 **A 程序的断点地址**和**当前 A 的 PSW** 压入堆栈。
3. B 被打断时，同样将 **B 的断点地址**压入堆栈
4. 当 C 执行完毕后，C 的程序末尾有一条**中断返回指令（如 x86 里的 `IRET`）。**这条指令会触发硬件去堆栈里**弹栈（Pop）**

* 硬件从栈顶弹出 B 的断点地址，CPU 的 PC（程序计数器）瞬间恢复到 B 被打断的地方，同时恢复 B 当时对应的屏蔽字（`110`）。**这就实现了回溯到 B**

5. B 接着执行完，再次执行 `IRET` 指令，硬件弹栈恢复 A 的断点和屏蔽字（`100`），**回溯到 A。**
6. A 执行完，执行 `IRET`，硬件弹栈恢复主程序的断点，**完美回溯到主程序**
